



Effective and Efficient One-pass Compression of Speech Foundation Models Using Sparsity-aware Self-pinching Gates

Haoning Xu¹, Zhaoqing Li¹, Youjun Chen¹, Huimeng Wang¹, Guinan Li¹, Mengzhe Geng², Chengxi Deng¹, Xunying Liu¹

¹The Chinese University of Hong Kong, Hong Kong SAR, China

²National Research Council Canada, Canada

hnxu@se.cuhk.edu.hk, xyliu@se.cuhk.edu.hk

Abstract

This paper presents a novel approach for speech foundation models compression that tightly integrates model pruning and parameter update into a single stage. Highly compact layer-level tied self-pinching gates each containing only a single learnable threshold are jointly trained with uncompressed models and used in fine-grained neuron level pruning. Experiments conducted on the LibriSpeech-100hr corpus suggest that our approach reduces the number of parameters of wav2vec2.0-*base* and HuBERT-*large* models by 65% and 60% respectively, while incurring no statistically significant word error rate (WER) increase on the test-clean dataset. Compared to previously published methods on the same task, our approach not only achieves the lowest WER of 7.05% on the test-clean dataset under a comparable model compression ratio of 4.26x, but also operates with at least 25% less model compression time.

Index Terms: speech recognition, model pruning, speech foundation models

1. Introduction

In recent years, advancements in self-supervised learning (SSL) for speech technologies, particularly through foundation models like wav2vec2.0 [1], HuBERT [2] and WavLM [3], have greatly improved their utility in applications like automatic speech recognition (ASR). Despite these advancements, the widespread adoption of these models in resource-constrained and on-device environments is limited due to their substantial memory and computational demands.

To address this challenge, extensive research has explored diverse neural network compression methods for ASR tasks, including but not limited to: **1) low-bit quantization** approaches that reduce memory footprint by replacing floating-point weights with low-precision values [4–8]; and **2) architecture compression** methods that focus on reducing structural redundancy in models, such as low-rank matrix factorization [9–11], knowledge distillation [12–18] and model pruning [19–25]. Furthermore, larger model compression ratios can be achieved by combining low-bit quantization and architecture compression [10, 26–28].

However, previous studies on model pruning of SSL-based ASR systems face the following limitations: **1) Significant performance degradation** is often observed as an increase in WER after performing compression. Coarse-grained “structured” pruning methods, which operate at the level of larger structures (e.g., channels) rather than individual parameters, may inadvertently remove critical parameters, resulting in performance degradation [22, 25, 29, 30]. It is also important to note that most studies failed to clearly define the criterion, such as statistical significance tests [31], to differentiate “acceptable” and

“unacceptable” performance loss due to compression. **2) Inconsistency between model pruning and parameter update** creates two separate and disjointed stages during compression, often leading to a large performance degradation. Approaches adopted in [21, 24, 32, 33] employ sequential paradigms that optimize pruning masks and parameters separately, which decouple pruning from parameter optimization in the ASR task. For example, the parameter pruning in [21] is driven by an independent layer-wise evaluation before post-pruning fine-tuning. **3) Substantial training time** inevitably arises from post-pruning refinement stages, including knowledge distillation [29], fine-tuning [21], and iterative pruning [24, 27, 32–34], which not only prolong development cycles but also complicate experimental workflows due to multi-stage requirements. **4) Excessive pruning-required parameter overhead**, such as: **i)** the extra parameters in KD-based pruning [22, 29], **ii)** the use of candidate architecture-specific weights in [10, 22, 25, 29] and **iii)** the parametrization of entire masks as trainable matrices [35].

To this end, this paper proposes a novel compression approach for SSL speech foundation models that tightly integrates model pruning and parameter update into a single stage, referred to as the *one-pass* stage. In each layer, the sparsity-aware self-pinching gate generates a differentiable pruning probability for each parameter by comparing its magnitude with the learnable threshold. For wav2vec2.0-*base* and HuBERT-*large*, only $6 \times 12 = 72$ and $6 \times 24 = 144$ additional parameters are introduced as thresholds, respectively¹.

Experiments conducted on Librispeech dataset suggest that the pruned wav2vec2.0-*base* and HuBERT-*large* using our method **1)** significantly outperform both **i)** uniform magnitude-based pruning where pruning and parameter update are separated, and **ii)** neural architecture search (NAS) [36] based channel-wise pruning, which introduces additional parameters for coarse-grained pruning, particularly when the overall sparsity reaches 50% or greater, and **2)** further achieve lossless² compression with sparsity of 65% and 60% on the test-clean subset, respectively. It should be noted that the time to compress wav2vec2.0-*base* and HuBERT-*large* only takes 11 and 13 GPU hours, respectively. Compared with prior pruning methods for SSL-based ASR systems, the proposed method obtains the lowest WER of 7.05% on the test-clean subset, while also reducing fine-tuning time with the model size limited to 23M parameters under a model compression ratio of 4.26x.

The main contributions of this paper include:

1) Our method reduces the fragility to pruning by using the

¹Compared to the uncompressed wav2vec2.0-*base* and HuBERT-*large* models, the additional pruning-required parameters account for only 8e-7 and 5e-7 of the total model size, respectively.

²“lossless” in this paper refers to no statistically significant WER increase against the uncompressed baseline.

proposed layer-level sparsity-aware self-pinching gate. Compared to the coarse-grained pruning approaches [22, 25, 29, 30], our fine-grained method assigns a specific pruning probability to each parameter, leading to better ASR performance.

2) Our method ensures consistency between model pruning and parameter update by integrating them into a single stage for SSL-based ASR systems. In contrast, previous fine-grained approach [21] performed pruning and fine-tuning separately. This one-pass compression stage also enables different layers to be pruned in varying sparsity based on their sensitivities, thereby achieving optimal mixed-sparsity assignments.

3) Our method demonstrates efficiency in terms of compression time, as it eliminates the need for additional operations such as post-pruning fine-tuning [21] or iterative pruning [32] after the one-pass compression stage.

4) Our method guarantees the compactness of pruning-required parameters by introducing a single threshold as an additional component for each layer. In contrast, the number of additional pruning-required parameters in previous methods is based on **i)** the design of teacher-student model [22, 29], **ii)** the number of candidates [10, 25] or **iii)** the layer size [35].

2. wav2vec2.0 and HuBERT Models

Speech SSL models such as wav2vec2.0 [1], HuBERT [2], and WavLM [3] share similar Transformer backbones. For example, HuBERT consists of a CNN encoder, a Transformer encoder, a projection layer and a code embedding layer. Transformer encoder accounts for over 90% of the total number of parameters, where each encoder block contains an MHSA module and an FFN module. In this work, we fine-tuned wav2vec2.0-*base* and HuBERT-*large* with a pure CTC decoder and pruned parameters in the 6 linear layers of each Transformer encoder block.

3. Previous works

3.1. Uniform Magnitude-based Pruning

The Uniform Magnitude-based Pruning (UMP) is inspired by the principle that parameters with smaller **magnitudes (absolute values)** have less influence on the output. As shown in Fig. 1 (a), in UMP, the parameters of a specific layer are sorted by magnitude, and the same proportion of parameters with relatively small magnitude is pruned across all layers, leading to consistent sparsity throughout the network.

However, UMP enforces isotropic sparsity across layers, ignoring layer-specific sensitivity profiles. While some layers tolerate aggressive pruning with little accuracy loss, others degrade significantly even with conservative sparsity.

3.2. NAS-based Channel-wise Pruning

A **channel** is defined as a row ($index, :$) or column ($[:, index]$) of the weight matrix of a linear layer. For example, in the p -th Transformer block, **1)** the m -th channels in the weight matrices of Multi-Head Self-Attention module, denoted as $\{\mathbf{Query}_{m,:}^p, \mathbf{Key}_{m,:}^p, \mathbf{Value}_{m,:}^p, \mathbf{Out}_{:,m}^p\}$, are simultaneously pruned and **2)** the n -th channels in the weight matrices of Feed-Forward Network module, denoted as $\{\mathbf{FFN.1}_{n,:}^p, \mathbf{FFN.2}_{n,:}^p\}$, are pruned together. The channels are sorted by the **sum of L2-magnitude**, which is calculated as $(\|\mathbf{Q}_{m,:}^p\|_2 + \|\mathbf{K}_{m,:}^p\|_2 + \|\mathbf{V}_{m,:}^p\|_2 + \|\mathbf{O}_{:,m}^p\|_2)$ and $(\|\mathbf{FFN.1}_{n,:}^p\|_2 + \|\mathbf{FFN.2}_{n,:}^p\|_2)$ for the m -th and n -th channels, respectively. Here $\|\cdot\|_2$ indicates the L2-norm.

In the Gumbel-Softmax differentiable neural architecture search (DARTS) [36, 37], we train a supernet that simultaneously contains 7 **candidate masks** per layer, as

shown in Fig. 1 (b), where the channels with relatively small **sum of L2-magnitude** are pruned in proportions of $\{0\%, 25\%, 50\%, 75\%, 87.5\%, 90\%, 92.5\%\}$ of the total channels, respectively. Specifically, the masked weight matrix of the l -th layer $\hat{\mathbf{W}}^l$ can be computed as follows:

$$\hat{\mathbf{W}}^l = \sum_{i=1}^7 \lambda_i^l \cdot \bar{\mathbf{M}}_i^l \odot \mathbf{W}^l, \quad (1)$$

where $\mathbf{W}^l$ is the weight matrix, $\bar{\mathbf{M}}_i^l$ denotes the i -th candidate mask and λ_i^l is the parameter that measures its importance. Gumbel-softmax is obtained by Gumbel sampling the output of the Softmax function, which is given by:

$$\lambda_i^l = \frac{\exp((\log \alpha_i^l + G_i^l)/T)}{\sum_{j=1}^7 \exp((\log \alpha_j^l + G_j^l)/T)}, \quad (2)$$

where α_i^l is an **architecture-dependent parameter** determining their contribution during search. $G_i^l = -\log(-\log(U_i^l))$ is the Gumbel variable, T is a hyperparameter and U_i^l is a uniform random variable. Let $\|\cdot\|_0$ indicate the number of non-zero parameters, the training loss is designed as:

$$\mathcal{L} = \mathcal{L}_{ctc} + \eta \sum_{l=1}^7 \sum_{i=1}^7 \lambda_i^l \cdot \|\bar{\mathbf{M}}_i^l\|_0, \quad (3)$$

where $\mathcal{L}_{ctc}$ is the CTC loss of the pruned system with the overall sparsity dynamically searched in real time and η is a constant coefficient that controls the overall sparsity.

While NAS-based techniques ensure consistency between model pruning and parameter update, they introduce considerable computational and parameter overhead. Furthermore, their effectiveness is highly dependent on the design of candidates.

4. Sparsity-aware Self-pinching Gates

The core concept of **Sparsity-aware Self-pinching Gates** is to leverage the weights already being learned to construct the mask using only one additional learnable threshold per layer. Our approach facilitates flexible allocation of layer-wise sparsity across different layers based on their sensitivities, ultimately enabling the model to reach a specific size limit.

For the l -th layer, $\mathbf{M}_{i,j}^l$ of the element-wise soft mask matrix $\mathbf{M}^l$ is expressed as:

$$\mathbf{M}_{i,j}^l = \text{Sigmoid} \left(\frac{(\mathbf{W}_{i,j}^l)^2 - (t^l)^2}{\tau} \right), \quad (4)$$

where t^l is a learnable threshold shared across all parameters in the weight matrix $\mathbf{W}^l$ of the l -th layer. Here, i and j denote the indices of parameters, and τ is a positive hyperparameter. Let $|\cdot|$ denote the number of parameters, the sparsity s^l of the l -th layer is defined as:

$$s^l = 1 - \frac{\|\mathbf{M}^l\|_0}{|\mathbf{M}^l|}. \quad (5)$$

The Straight-Through Estimator (STE) [38] is used during the one-pass fine-tuning stage: **1)** During forward propagation or inference, $\mathbf{M}^l$ is rounded to a binary mask $\bar{\mathbf{M}}^l$ and the masked weight matrix is computed as $\bar{\mathbf{W}}^l = \mathbf{W}^l \odot \bar{\mathbf{M}}^l$. Specifically, the parameters in $\mathbf{W}^l$ with magnitude exceeding the threshold t^l will be retained, while those below the threshold will be pruned, which can be expressed as:

$$\bar{\mathbf{M}}_{i,j}^l = \begin{cases} 1 & (\mathbf{W}_{i,j}^l)^2 \geq (t^l)^2 \\ 0 & (\mathbf{W}_{i,j}^l)^2 < (t^l)^2 \end{cases}, \quad (6)$$

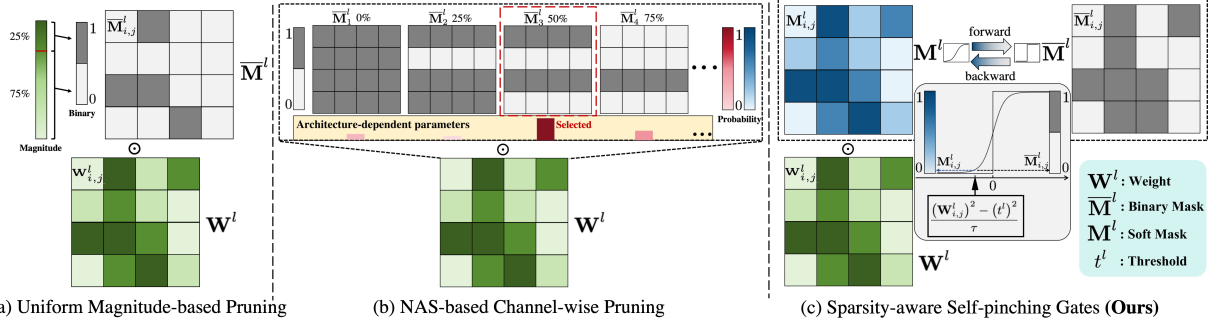


Figure 1: Comparison between Uniform Magnitude-based Pruning (UMP), NAS-based Channel-wise Pruning (NAS-CP) and Sparsity-aware Self-pinching Gates (ours). For the l -th layer, (a) UMP directly prunes the same proportion of parameters by magnitude across all layers; (b) NAS-CP introduces architecture-dependent parameters proportional to architecture candidates, which are pre-selected before NAS search; (c) Ours utilizes the weights that are already being learned to construct the mask with only one additional threshold.

2) During backward propagation, the gradient is accumulated using the actual values in $\bar{M}^l$. The training loss is given by:

$$\mathcal{L} = \mathcal{L}_{ctc} + \eta \sum_l \|\bar{M}^l\|_0, \quad (7)$$

where η is a constant coefficient to control the overall sparsity. η is set to 0 if the desired sparsity is achieved; otherwise, it remains at its preset value.

Our method shown in Fig. 1 (c) simultaneously optimizes the threshold t^l and the weight matrix $\mathbf{W}^l$, thus mitigating the mismatch between model pruning and parameter update. The above fine-grained neural level pruning is more powerful in mitigating performance loss due to model compression than coarse-grained pruning methods, which are applied at the channel level and easier to implement.

5. Experiments

5.1. Experimental setup

Uncompressed baselines and data. For wav2vec2.0-base, the wav2vec2-base-100h is downloaded from Huggingface³ as our baseline. For HuBERT-large, we fine-tuned HuBERT-large-ll60k⁴ for 20 epochs as our baseline, with other setups consistent with those in **One-pass pruning and fine-tuning**. All systems are trained on LibriSpeech’s [39] 100-hour clean set.

One-pass pruning and fine-tuning. We utilized the AdamW optimizer with a learning rate of $2e-4$ and a batch size of 32 for both wav2vec2.0 and HuBERT systems. A linear warmup is implemented for the first 10% of the training steps, followed by a linear decay to zero. All pruned wav2vec2.0-base and HuBERT-large systems are obtained by fine-tuning 30 and 10 epochs from the baselines of wav2vec2.0-base and HuBERT-large, respectively. The threshold t in each layer is initialized to $1e-5$. T and τ are cosine-annealed from 0.5 to 0.01. All experiments are conducted on a single NVIDIA A40 (48 GB).

5.2. Main results

To facilitate comparison, we implemented Uniform Magnitude-based Pruning (UMP), NAS-based Channel-wise Pruning (NAS-CP)⁵ and our method (Ours)⁶, as shown in Fig. 2.

³Huggingface: facebook/wav2vec2-base-100h

⁴Huggingface: facebook/HuBERT-large-ll60k

⁵Empirically, in NAS-CP, η is set to $4e-5$ and $3e-5$ for wav2vec2.0-base and HuBERT-large, respectively, when the desired sparsity is less than 75%; otherwise, it is set to $2e-4$ and $5e-5$.

⁶Empirically, in Ours, η is set to $2e-5$ and $1e-6$ for wav2vec2.0-base and HuBERT-large, respectively, when the desired sparsity is less than 65%; otherwise, it is set to $3e-5$ and $2e-6$.

Table 1: WER(%) of pruned wav2vec2.0-base and HuBERT-large with different sparsity, using fine-tuning epochs of 30 and 10, respectively. Only selected key results are shown here. † means it has no statistically significant (MAPSSWE [31], $\alpha=0.05$) WER increase with the corresponding baseline.

Sys.	Sparsity	# Params (M)	# Additional components	dev		test	
				clean	other	clean	other
Baselines							
1	wav2vec2.0-base	95.04	-	6.10	13.79	6.06	13.52
2	HuBERT-large	316.60		3.35	8.13	3.44	8.34
Pruned wav2vec2.0-base systems using Sparsity-aware Self-pinching Gates							
3	50%	51.93	72	5.97 [†]	16.09	6.00 [†]	16.06
4	60%	43.44		6.06 [†]	16.52	6.07 [†]	16.08
5	65%	39.19		6.03 [†]	16.97	6.12[†]	16.89
6	70%	34.94		6.21[†]	17.17	6.45	17.46
7	75%	30.70		6.41	17.82	6.71	18.08
8	80%	26.45		6.71	18.63	6.95	19.27
9	85%	22.20		7.49	20.47	7.46	20.74
10	90%	17.96		8.22	23.36	8.63	24.30
Pruned HuBERT-large systems using Sparsity-aware Self-pinching Gates							
11	50%	164.37	144	3.45[†]	9.14	3.51 [†]	8.99
12	60%	134.14		3.54	9.44	3.54[†]	9.33
13	65%	119.03		3.62	9.86	3.65	9.75
14	70%	103.92		3.76	10.27	3.76	10.15
15	75%	88.81		3.88	11.01	3.98	11.04
16	80%	73.70		4.19	12.51	4.36	12.39
17	85%	58.59		4.83	14.66	4.82	14.72
18	90%	43.48		6.11	18.91	6.37	19.29
Pruned HuBERT-large systems using Mixed-sparsity							
19	10%	285.25	-	3.35 [†]	8.29[†]	3.49 [†]	8.48 [†]
20	20%	255.03		3.43 [†]	8.32	3.43 [†]	8.42 [†]
21	30%	224.81		3.31 [†]	8.81	3.45 [†]	8.47[†]
22	50%	164.37		3.46 [†]	8.89	3.49 [†]	8.72
Pruned HuBERT-large systems using NAS-based Channel-wise Pruning							
23	50%	164.37	1008	7.67	20.99	7.79	21.34
Pruned HuBERT-large systems using Uniform Magnitude-based Pruning							
24	50%	164.37	-	3.49	8.85	3.64	8.83

5.2.1. Comparison with Uniform Magnitude-based Pruning

Pruned wav2vec2.0-base using both UMP and our method achieve lossless compression with all sparsity of below 50% on the clean subsets. Beyond 50%, pruned wav2vec2.0-base and HuBERT-large using ours consistently exceed those using UMP (Fig. 2 (1)-(4)) on all subsets, respectively. On the dev-clean subset, pruned wav2vec2.0-base and HuBERT-large using ours achieve lossless compression with maximum sparsity of 70% (Fig. 2 (1) and Sys. 6 in Tab.1) and 50% (Fig. 2 (2) and Sys. 11 in Tab.1), respectively, versus UMP’s 60% and 30% (Fig. 2 (1)-(2)). On the test-clean subset, ours achieves 65% (Fig. 2 (3) and Sys. 5 in Tab.1) and 60% (Fig. 2 (4) and Sys. 12 in Tab.1), compared to UMP’s 50% and 40% (Fig. 2 (3)-(4)).

We conjecture that the inferior performance of UMP is partially caused by the inconsistency that decouples pruning from parameter optimization. To verify this, we implement a decoupled version of our method, which involves pruning the parameters with relatively small magnitude from the uncompressed model to meet the layer-wise sparsity obtained from our one-

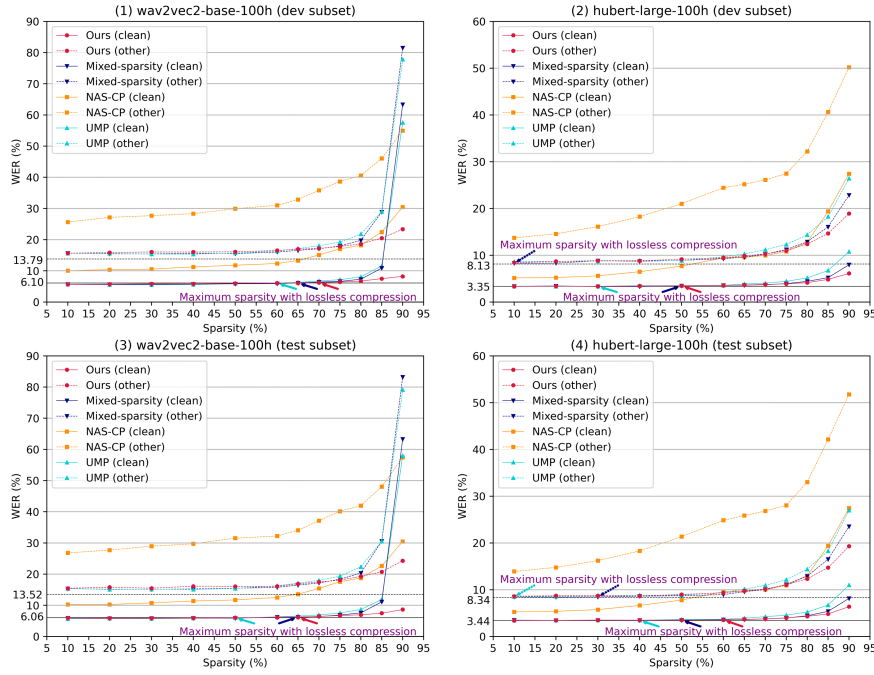


Figure 2: The ASR performance of the pruned wav2vec2-base-100h on the (1) dev and (3) test subsets, as well as the pruned hubert-large on the (2) dev and (4) test subsets with different sparsity using different methods. Abbreviations are the same as those in Figure 1. Color-matched arrows point to the maximum sparsity preserving lossless compression of the methods in corresponding color.

pass method, as shown in Fig. 2 under the label *Mixed-sparsity*. It outperforms UMP across most sparsity levels, suggesting that our one-pass method effectively accounts for the varying sensitivities of different layers. However, it performs worse than our one-pass method, highlighting that the inconsistency between pruning and parameter update negatively affects ASR performance. Notably, on the less-explored dev-other and test-other subsets, pruned HuBERT-large systems using *Mixed-sparsity* maintain lossless compression with maximum sparsity of 10% (Fig. 2 (2) and Sys. 19 in Tab.1) and 30% (Fig. 2 (4) and Sys. 21 in Tab.1), respectively, while UMP only achieves 10% on the test-other subset (Fig. 2 (4)).

Table 2: The ASR performance comparison of our method versus previous compression methods on the test-clean subset. The values in brackets “(” and “)” represent the transformer sparsity. Comp.ratio: the compression ratio relative to the uncompressed model. *: from leaderboard of SUPERB [40].

Sys.	System	Train set	# Params (M)	Comp. ratio	Training time & config			WER%
					Hours	Epochs	Iterations	
Baselines								
1	wav2vec2.0-base	100h	95.04					6.06
2	HuBERT-base		94.68	-	-	-	-	6.42*
3	WavLM-base+		94.70					5.59*
4	HuBERT-large		316.60					3.44
Prior Compression Methods								
5	DistilHuBERT [15]	960h	23.49	4.03	55	200	-	13.37*
6	FitHuBERT-100 [18]	100h	22.49	4.21	>12	100	-	12.66*
7	FitW2V2-100 [18]		4.23	-	-	-	-	14.77*
8	FitHuBERT-960 [18]	960h	22.49	4.21	>93.6	-	-	12.09*
9	FitW2V2-960 [18]		31.63	3.00	-	80	-	11.44*
10	12-Layer Half [17]	960h	26.87	3.52	-	-	200K	10.96*
11	DPHuBERT [29]	960h	23.59	4.01	-	-	-	10.47*
12	DPWavLM [29]		23.59	4.01	24	-	75K	10.19*
13	SKHuBERT [30]	960h	23.59	4.01	-	-	-	10.78*
14	SKWavLM [30]		23.51	4.03	24	-	75K	10.03*
15	Wang et al. [22]	960h	26.57	3.56	36	-	200K	10.29*
16	DKD LSTM HuBERT [41]	960h	18.80	5.04	-	-	200K	10.64*
17	SparseWAV2VEC2 (85%) [21]	100h	22.27	4.27	-	-	-	8.08
18	SparseWAVLM+ (85%) [21]		22.98	4.12	-	>40	-	7.12
19	SparseWAV2VEC2 (75%) [21]		-	-	-	-	-	7.11
Ours								
20	Pruned wav2vec2-base (85%)	100h	22.20	4.28	11	30	27K	7.46
21	Pruned wav2vec2-base (75%)		30.70	3.10	11	30	27K	6.71
22	Pruned WavLM-base+ (85%)	100h	22.21	4.26	11	30	27K	7.05
23	Pruned HuBERT-large (90%)	100h	43.48	7.28	13	10	9K	6.37

5.2.2. Comparison with NAS-based Channel-wise Pruning

Predictably, compared with NAS-CP that prunes multiple channels, our method applied to both wav2vec2.0-base and HuBERT-large achieves consistently lower WERs across all sparsity as demonstrated in Fig. 2 (1)-(4). Regarding the additional components, NAS-CP requires 7 architecture-dependent parameters per layer (e.g., 1008 total for HuBERT-large of Sys. 23 in Tab. 1) in our configurations (Sec. 3.2). In contrast, our method maintains only one threshold per layer (e.g., 144 total for HuBERT-large of Sys. 11 in Tab. 1).

5.3. Comparison with other compression methods

As illustrated in Tab. 2, our method (Sys. 20) achieves $\geq 2\%$ absolute WER reduction over Sys. 5-16 under equivalent model-size limits. In order to compare training times, we consolidated different types (Hours/Epochs/Iterations) of training-time metrics from previous works into standardized comparisons, where our method demonstrates consistent time-efficiency advantages. Notably, the WERs of Sys. 20 and Sys. 21 using our method surpass Sys. 17 and Sys. 19, respectively, while requiring at least 25% less fine-tuning epochs, highlighting the effectiveness and efficiency of our approach. We also pruned WavLM-base+ downloaded from Huggingface⁷ using the same setup as wav2vec2.0-base, yielding the lowest WER (Sys. 22 in Tab. 2) of 7.05% on the test-clean subset under the model-size constraint of 23M parameters, compared to Sys. 5-19 in Tab. 2.

6. Conclusion

We introduced a cutting-edge one-pass compression method that simultaneously prunes and trains SSL speech foundation models using a threshold per layer. Our method shows superior performance under the same model-size constraint while reducing the fine-tuning time compared to the previous works. We will further apply our methods to more datasets in future work.

⁷Huggingface: patrickvonplaten/wavlm-libri-clean-100h-base-plus

7. Acknowledgements

This research is supported by Hong Kong RGC GRF grant No. 14200220, 14200021, 14200324 and Innovation Technology Fund grant No. ITS/218/21.

8. References

- [1] A. Baevski, Y. Zhou, A. Mohamed *et al.*, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” in *NeurIPS*, 2020.
- [2] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai *et al.*, “HuBERT: Self-supervised speech representation learning by masked prediction of hidden units,” *IEEE/ACM T-ASLP*, vol. 29, pp. 3451–3460, 2021.
- [3] S. Chen, C. Wang, Z. Chen *et al.*, “WavLM: Large-scale self-supervised pre-training for full stack speech processing,” *IEEE J-STSP*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [4] O. Rybakov, P. Meadowlark, S. Ding *et al.*, “2-bit conformer quantization for automatic speech recognition,” in *Interspeech*, 2023.
- [5] S. Ding, P. Meadowlark, Y. He *et al.*, “4-bit conformer with native quantization aware training for speech recognition,” in *Interspeech*, 2022.
- [6] S. Kim, A. Gholami, Z. Yao *et al.*, “I-bert: Integer-only bert quantization,” in *International conference on machine learning*. PMLR, 2021, pp. 5506–5518.
- [7] H. Xu, Z. Li, Z. Jin *et al.*, “Effective and efficient mixed precision quantization of speech foundation models,” *arXiv preprint arXiv:2501.03643*, 2025.
- [8] E. Fish, U. Michieli, and M. Ozay, “A model for every user and budget: Label-free and personalized mixed-precision quantization,” in *Interspeech*, 2023.
- [9] S. Li, M. Xu, and X.-L. Zhang, “Efficient conformer-based speech recognition with linear attention,” in *APSIPA ASC*, 2021.
- [10] Z. Li, T. Wang, J. Deng *et al.*, “Lossless 4-bit quantization of architecture compressed conformer asr systems on the 300-hr switchboard corpus,” in *Interspeech*, 2023.
- [11] H. Wang and W.-Q. Zhang, “Unstructured pruning and low rank factorisation of self-supervised pre-trained speech models,” *IEEE Journal of Selected Topics in Signal Processing*, 2024.
- [12] J. Rathod, N. Dawlatabad, S. Singh *et al.*, “Multi-stage progressive compression of conformer transducer for on-device speech recognition,” in *Interspeech*, 2022.
- [13] J. Park, S. Jin, J. Park *et al.*, “Conformer-based on-device streaming speech recognition with KD compression and two-pass architecture,” in *IEEE SLT*, 2023.
- [14] Y. Fu, Y. Kang, S. Cao *et al.*, “DistillW2V2: A small and streaming wav2vec 2.0 based asr model,” *arXiv preprint arXiv:2303.09278*, 2023.
- [15] H.-J. Chang, S.-w. Yang, and H.-y. Lee, “DistilHuBERT: Speech representation learning by layer-wise distillation of hidden-unit bert,” in *ICASSP*, 2022.
- [16] R. Wang, Q. Bai, J. Ao *et al.*, “LightHuBERT: Lightweight and configurable speech representation learning with once-for-all hidden-unit bert,” in *Interspeech*, 2022.
- [17] T. Ashihara, T. Moriya, K. Matsuura *et al.*, “Deep versus wide: An analysis of student architectures for task-agnostic knowledge distillation of self-supervised speech models,” in *Interspeech*, 2022.
- [18] Y. Lee, K. Jang, J. Goo *et al.*, “FitHuBERT: Going thinner and deeper for knowledge distillation of speech self-supervised learning,” in *Interspeech*, 2022.
- [19] Z. Wu, D. Zhao, Q. Liang *et al.*, “Dynamic sparsity neural networks for automatic speech recognition,” in *ICASSP*, 2021.
- [20] J. Lee, J. Kang, and S. Watanabe, “Layer pruning on demand with intermediate etc,” in *Interspeech*, 2021.
- [21] T. Gu, B. Liu, H. Shao *et al.*, “Sparsewav: Fast and accurate one-shot unstructured pruning for large speech foundation models,” in *Proc. Interspeech 2024*, 2024, pp. 4498–4502.
- [22] H. Wang, S. Wang, W.-Q. Zhang *et al.*, “Task-agnostic structured pruning of speech representation models,” in *Interspeech 2023*, 2023, pp. 231–235.
- [23] H. Jiang, L. L. Zhang, Y. Li *et al.*, “Accurate and structured pruning for efficient automatic speech recognition,” in *Interspeech*, 2023.
- [24] V. S. Lodagala, S. Ghosh, and S. Umesh, “PADA: Pruning assisted domain adaptation for self-supervised speech representations,” in *IEEE SLT*, 2023.
- [25] Y. Peng, K. Kim, F. Wu *et al.*, “Structured pruning of self-supervised pre-trained models for speech recognition and understanding,” in *ICASSP*, 2023.
- [26] Z. Li, H. Xu, T. Wang *et al.*, “One-pass multiple conformer and foundation speech systems compression and quantization using an all-in-one neural model,” in *Interspeech 2024*, 2024, pp. 4503–4507.
- [27] N. Wang, C.-C. C. Liu, S. Venkataramani *et al.*, “Deep compression of pre-trained transformer models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 14 140–14 154, 2022.
- [28] S. Ding, Q. David, D. Rim *et al.*, “USM-Lite: Quantization and sparsity aware fine-tuning for speech recognition with universal speech models,” in *ICASSP*, 2024.
- [29] Y. Peng, Y. Sudo, S. Muhammad *et al.*, “DPHuBERT: Joint distillation and pruning of self-supervised speech models,” in *Interspeech*, 2023.
- [30] L. Zampierin, G. B. Hacene, B. Nguyen *et al.*, “Skill: Similarity-aware knowledge distillation for speech self-supervised learning,” in *2024 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops (ICASSPW)*, 2024, pp. 675–679.
- [31] L. Gillick and S. J. Cox, “Some statistical issues in the comparison of speech recognition algorithms,” in *ICASSP*, 1989.
- [32] C.-I. J. Lai, Y. Zhang, A. H. Liu *et al.*, “Parp: Prune, adjust and re-prune for self-supervised speech recognition,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 21 256–21 272, 2021.
- [33] M. Yang, A. Tjandra, C. Liu *et al.*, “Learning asr pathways: A sparse multilingual asr model,” in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [34] S. Wang, H. Wang, J. Li *et al.*, “Learnable sparsity structured pruning for acoustic pre-trained models,” in *Proceedings of the 2023 6th International Conference on Signal Processing and Machine Learning*, 2023, pp. 76–81.
- [35] Y. Fu, Y. Zhang, K. Qian *et al.*, “Losses can be blessings: Routing self-supervised speech representations towards efficient multilingual and multitask speech processing,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 20 902–20 920, 2022.
- [36] H. Liu, K. Simonyan, and Y. Yang, “Darts: Differentiable architecture search,” in *International Conference on Learning Representations*, 2018.
- [37] C. J. Maddison, A. Mnih, and Y. W. Teh, “The concrete distribution: A continuous relaxation of discrete random variables,” in *International Conference on Learning Representations*, 2022.
- [38] Y. Bengio, N. Léonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” *arXiv preprint arXiv:1308.3432*, 2013.
- [39] V. Panayotov, G. Chen, D. Povey *et al.*, “LibriSpeech: an asr corpus based on public domain audio books,” in *ICASSP*, 2015.
- [40] S. wen Yang, P.-H. Chi, Y.-S. Chuang *et al.*, “Superb: Speech processing universal performance benchmark,” in *Interspeech 2021*, 2021, pp. 1194–1198.
- [41] D. de Oliveira and T. Gerkmann, “Distilling HuBERT with LSTMs via decoupled knowledge distillation,” in *ICASSP*, 2024.